

Amit Joshi

Web/Mobile Senior Lab

Period 6 Kosek

18 May 2018

Encoding hidden information in image files using Steganography

Abstract:

Encoding hidden information in images is widely used in confidential communication and secret data storing. Sensitive information stands a much better chance of going undetected if it is hidden in such a way that from the outside, it doesn't appear to be sensitive information at all. Hiding this sensitive information in images conceals the existence of the information itself. If the sensitive information is text, the information can be encrypted to further conceal the information. Creating a web app for Steganography allows it to be more accessible.

Introduction

Steganography comes from the words for “hidden” and “writing” in greek. Hence, steganography is the practice of encoding hidden information inside a vessel. In this case, the vessel is an image and the hidden information is stored by slightly modifying the image's pixels. The objective of this project is to create a website that allows users to upload a carrying image and hidden information and then be able to decode the hidden information later. This allows users to keep secret information free of detection by encoding the hidden information within the

image. The user can now send the image with the hidden information to the intended recipient or save it for future use. I was motivated to create a web app for steganography because of the lack of robust, secure, and free online steganography tools available. In most cases, free websites use a kind of steganography that is very easy to crack due to the lack of a password or key, among others. My objective is to make a web app for keyed steganography which uses other security measures that will be discussed later on.

Background

While cryptography encodes a message in a way that it is impossible to decode the message without a key or cipher, steganography aims to stop hidden information from being detected altogether. Due to the ability of steganography to avoid suspicion, steganography can even be used in images that are in plain sight. It has been suspected that steganography has been used by terrorist organizations to send messages to one another (Rice University). It makes sense then, that from a defensive standpoint, it is important to be able to recognize and then decode hidden messages by malicious organizations.

At its core, steganography provides another level of security over encryption. Not only can it hide the existence of sensitive information, but it can also enhance the security of already encrypted data (Kawaguchi). This means that data that is already encrypted using cryptography can then be the hidden information that is encoded into an image using steganography. So, instead of hiding regular text into an image, one can instead hide encrypted text into the image to increase the security of the hidden information immensely. This is because if a steganography decoder were run on an image carrying encrypted text, all that would be returned is jumbled, encrypted text. It is very difficult for someone to decide whether to even bother to decrypt the

jumbled, encrypted text since similar text would be generated if the steganography decoder was run on a normal image.

For both watermarking (image-in- image encoding) and text-in- image encoding, encoding hidden information in files can be done by modifying the pixel colors of the image so slightly that it is impossible for a human to detect any modification. This can be done by modifying the least significant bits of each pixel channel (red, green, and blue) of the image. If each pixel has three values, then three bits of information can be encoded in each pixel (Rice University). This is called the Least Significant Bit (LSB) Algorithm. This ensures that the image is modified so slightly that a passerby will not notice that the image contains secret data. In fact, even when it is known that a particular image contains secret data, it is virtually impossible for a human to notice anything different from the original vessel image. This is because each pixel channel has a value from 0 to 255, and the most each channel can change is by 1. Hence, the maximum change is $1/255$ which is approximately equal to 0.39 percent difference.

A successful application of steganography is one that is not noticeable. It must modify the pixels so slightly that the image will look innocuous so people will not look twice at it. Modifying the pixel values of an image can be done by using Pillow, a fork for PIL (Python Image Library). The reason Pillow will be used for this project is that Pillow supports Python 3. Furthermore, Pillow's Image class can be used to open an image, load its pixel values, and get the dimensions of the image.

To implement my project, I decided to create a Node.js server. The server had two methods that would either run a python script for encoding or run one for decoding using the

npm package Child Process. The availability of Child Process made it easy to pass arguments to the python script from node.js. The Child Process would spawn the python synchronously and receive feedback in about three to four seconds for the encoding script and about a quarter of a second for the decoding script. For encoding, the server returns a string containing the name of the file with the secret messages encoded to the html page and then the html page makes a subsequent call to actually retrieve the image. This then would be automatically downloaded to the user's computer by creating an invisible link element and then "clicking" it programmatically. For the decoding, the server simply returns the decoded message as a string.

The interface of my web app will ask the user to upload a carrying image and then hidden information to be encoded into the carrying image. The user can choose whether to hide text or hide an image. As long as the hidden information size does not exceed 25% of the carrying image size, the website will automatically send the carrying image and the hidden information to a server. If the hidden information size does exceed this limit, the user will be notified of this potential problem before the website sends the carrying image and hidden information to the server. Once the Node.js server gets the information, it will run the encoding script. Once this is complete, the server sends back the encoded image for the user to download. Another user can then use the decoder on the website which does the opposite process, as long as that user has the password.

Now, my server has to be prepared to deal with multiple users trying to use my web app at once. To do this, the user's vessel image upload needs to be renamed with a collision-resistant system. In other words, two images uploaded should not have the same file name. I

accomplished this by taking the time in milliseconds since the start of the Unix Epoch (January 1st, 1970) and combined that with the file extension.

Some struggles associated with the website was the uploading of the image. Eventually, I found out that the way to do it was use npm's express-fileupload on the server. This could be used after the client put the image into a FormData object and sent that in the AJAX request. After this, the image would be saved onto my server to be used by the Python 2 script.

Development/Techniques

It was very quick to get a basic application of steganography working, where the pixels are modified in a line from left to right. Even though it would have been difficult for a human to detect the existence of the secret message in this case, I decided to implement further security measures. First, I implemented keyed steganography. The way this works is that while a user who wishes to encode a message also inputs a password. I then modify this password to create a random seed which I will then use to determine the order in which the pixels need to be modified to encode the secret message. While other steganographic methods often encode the secret message along the first row of pixels of the image, this is far too easy to crack. With this added security measure, only an individual with a key would be able to read the pixels in the correct order to accurately decode the message. Next, I focused on obfuscating the message itself. The way steganography works is that it converts the secret message into a series of zeros and ones. If one figures out the correct series of zeros and ones, it is possible to decode the secret message. This is a potential problem as outsiders may, in certain situations, be able to find the correct series. I solved this problem by taking the absolute value of the derivative of the sequence a

particular number of times. For example, the sequence “011” would become “010” when applying this function one time. I used the random seed that I had created to determine how many times I should apply this function to get the final string to be encoded. For decoding, the message would be integrated to get the original series of zeros and ones. My next security measure was the addition of random noise. This means that random pixels that have nothing to do with the image get modified in the hopes of confusing outsiders. To find out which pixels should be randomly modified, I took the remaining pixels that the encoding hadn’t touched, and made each pixel subject to a random number generated from 0 to 1. If the random number was less than .001, the pixel would be randomly modified. I found this number to be optimal because this is actually the slowest part of my steganography process, and if the number were to be any larger, it would be too slow for practical use.

Amit Joshi Senior Research Project

Steganography

Encoding

Enter message to be encoded:

Enter password, if you wish to:

Choose vessel image: No file chosen

Submit

Reset

Decoding

Enter password, if you wish to:

Submit

Reset

Discussion/Conclusion

It was found that most of the security techniques such as the LSB algorithm, the scattering of the message, and the encryption of the text beforehand were nearly instant.

However, the addition of the random noise took roughly four seconds for a 500x500 vessel image. I decided that it was still a good idea to keep it due to the fact that it can throw off anyone that tries to figure out my algorithm. One future considerations may be to reduce the frequency of the random noise. Currently, the random noise occurs at roughly 0.1% of the pixels that haven't already been modified. If this were to be lowered, the time needed for random noise addition would be lowered proportionally.

Works Cited

Kawaguchi, Eiji. "Applications of Steganography." *Applications of Steganography*. Eiji

Kawaguchi, 14 Apr. 2017. Web.

Rice University. "What Is Steganography." *What Is Steganography*. Rice University, n.d.

Web.

William_Wilson. "Steganography - Java Tutorials | Dream.In.Code." : *Programming &*

Web Development Community. Dreamincode, 11 May 2007. Web.